

ANNA UNIVERSITY CHENNAI-600025

UNIVERSITY COLLEGE OF ENGINEERING DINDIGUL

OCS-352-IOT CONCEPTS AND APPLICATIONS

ASSIGNMENT-II

MINI PROJECT

GOOGLE ASSISTANT CONTROL LED LAMP USING ESP32

SUBMITTED BY

NAME: 1. D. KAVIYA

2. S. RAGA VARSHINI

REG NO: 1. 922322105016

2.922322105025

DEPARTMENT OF ELECTRICAL AND ELECRONICS ENGINEERING

ACKNOWLEDGEMENT

We extend our warm gratitude to our Dean**, Dr.S.SUTHA, M.E., Ph.D.,** who is always with us in all our activities that we have in my college premises.

We thank our Head of the Department and the project coordinator, **Dr.M.SENTHILKUMAR, M.E., Ph.D.,** for his graceful support to complete this project.

We extend our gratitude to my supervisor, **Dr. M.SENTHILKUMAR, M.E., Ph.D.,** faculty in Electrical Engineering Department, for his valuable guidance encouragement, constructive and unreserved co-operation extended at each stage to complete this project successfully.

Last, we are extremely grateful in our parents and friends for their constant encouragement and moral support throughout our venture.

ABSTRACT

This project demonstrates the implementation of a voice-controlled lighting system using the Sinric Pro IoT platform. By integrating an ESP32 microcontroller with a relay module, the system enables users to control electrical lights through voice commands via Amazon Alexa or Google Assistant. Sinric Pro acts as a bridge between the cloud and the ESP32, allowing seamless communication and control over the internet. The primary goal of this project is to offer a low-cost, user-friendly, and efficient solution for smart home automation. This system enhances convenience, especially for elderly or physically challenged individuals, by eliminating the need for manual switching. The project also serves as a foundation for more complex home automation systems, showcasing how IoT and cloud services can be leveraged for practical, everyday applications.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
I	ABSTRACT	iii
II	TABLE OF CONTENTS	iv
III	LIST OF FIGURES	v
IV	LIST OF ABBREVIATIONS	vi
1	INTRODUCTION	1
2	PROJECT OVERVIEW-BLOCK DIAGRAM	3
2.1	SYSTEM ARCHITECTURE	5
2.2	COMPONENTS USED	7
3	HARDWARE DESIGN-CIRCUIT DIAGRAM	9
3.1	ESP32/NODEMCU CIRCUIT	12
3.2	RELAY AND LED INTERFACING	14
4	SOFTWARE IMPLEMENTATION	17
4.1	SINRIC PRO SETUP	22
4.2	GOOGLE ASSISTANT INTEGRATION	28
5	CODE FLOW AND LOGIC	29
6	PROGRAMMING AND IMPLEMENTATION	31
7	RESULT AND DISCUSSION	35
8	CONCLUSION	36
9	REFERENCES	37

LIST OF FIGURES

CHAPTER NO	TITLE	PAGE NO
2	Project overview-block diagram	3
2.1	System architecture	5
2.2	Components used	7
3	Hardware design-circuit diagram	9
3.1	ESP32/NODEMCU circuit	12
3.2	Relay and LED interface	14
4	Software implementation	17
4.1	Sinric pro setup	22
4.2	Google assistant integration	28

LIST OF ABBREVATION

ABBREVATION	FULL FORM
IOT	INTERNET OF THINGS
LED	LIGHT EMITTING DIODE
ESP32	ESPRESSIF SYSTEMS 32 BIT MICROCONTROLLER
API	APPLICATION PROGRAMMING INTERFACE
WIFI	WIRELESS FIDELITY
GPIO	GENERAL PURPOSE INPUT/OUTPUT
IDE	INTEGRATED DEVELOPMENT ENVIRONMENT

CHAPTER 1

INTRODUCTON

With the rapid growth of Internet of Things (IoT), smart home automation has become more accessible and practical for everyday use. This project focuses on building a voice-controlled light system using Sinric Pro and Google assistant. By combining a microcontroller like the ESP32, a relay module, and Sinric Pro's cloud services, users can control a light simply by speaking a command to google assistant (e.g., "hey google, turn on the light").

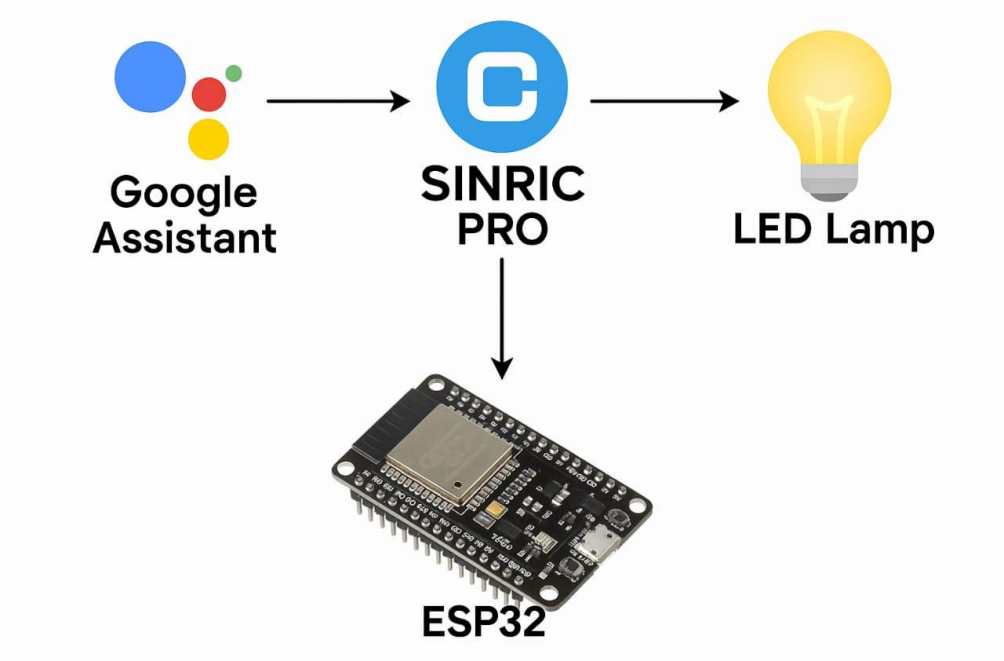
Sinric Pro acts as a bridge between the microcontroller and Google assistant, handling the communication over the internet securely and reliably. This setup not only enhances convenience but also promotes energy efficiency and supports hands-free control, which is especially useful for the elderly or people with physical disabilities.

This project is a great starting point for understanding how IoT, cloud platforms, and voice assistants can be integrated to build practical smart home applications.

CHAPTER 2

PROJECT OVERVIEW-BLOCK DIAGRAM

The Internet of Things (IoT) has revolutionized the way electronic devices are controlled and monitored. This project aims to design and implement a smart home automation system that allows the user to control an LED lamp using Google Assistant voice commands. The control is achieved through Sinric Pro, an IoT platform that bridges voice services like Google Assistant with Wi-Fi-enabled microcontrollers such as the NodeMCU (ESP8266 or ESP32).



The system architecture involves connecting the LED (or lamp)

to a relay module which is then interfaced with the NodeMCU.

Sinric Pro handles communication between Google Assistant and the NodeMCU via cloud services. When a voice command like "Turn on the lamp" is issued, it is processed through Google Assistant and routed to Sinric Pro, which then updates the device state. The NodeMCU reads this state change and switches the LED ON or OFF accordingly.

This project highlights the integration of voice recognition, cloud services, and microcontroller-based hardware control, providing a low-cost and scalable solution for smart home applications.

2.1 SYSTEM ARCHITECTURE

Controlling an LED lamp using Google Assistant and ESP32 via Sinric Pro.



System Architecture

1. User Interface Layer

Google Assistant

Accepts voice commands like “Turn on the lamp.”

Sends the request to Sinric Pro cloud.

2. Cloud Service Layer

Sinric Pro Cloud Platform

Receives commands from Google Assistant

Authenticates and routes the command to the correct device

Communicates with the ESP32 through WebSockets (over the internet)

3. Device Control Layer

ESP32

Stays connected to Sinric Pro via WebSocket

Receives command (e.g., turn ON/OFF)

Controls GPIO pin to switch the LED or lamp (directly or through a relay)

4. Actuator Layer

LED Lamp

Turns ON/OFF based on signal from ESP32

2.2 COMPONENTS USED

Here are the main components:



Hardware Components:

1. ESP32 Development Board

Microcontroller with built-in Wi-Fi

Acts as the main controller

2. LED Lamp or Relay Module

LED for testing

Relay module to control a real lamp (AC)

3. Power Supply

USB power for ESP32

External power if using relay with high-power lamp

4. Connecting Wires / Bread board

Software Components:

1. Arduino IDE

For programming the ESP32

2. Sinric Pro Platform

Provides cloud service and device linking

3. Google Assistant

Voice interface to send commands

4. Sinric Pro Library (in Arduino IDE)

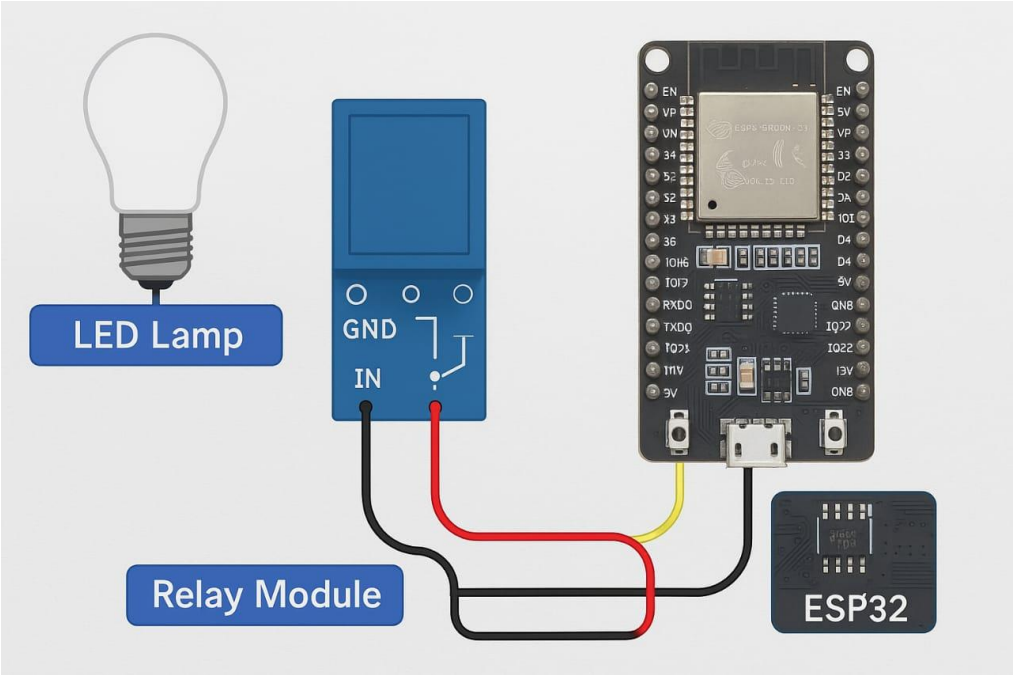
Handles WebSocket and communication

5. Wi-Fi Connection

For cloud communication between ESP32 and Sinric

CHAPTER 3

HARDWARE DESIGN-CIRCUIT DIAGRAM



Key Components:

1. ESP32 Board – Main controller that receives commands from Sinric Pro (linked with Google Assistant).

2. Relay Module (5V) – Acts as a switch to control high-voltage devices like an AC LED lamp.

3. LED Lamp – The load device to be controlled.

Wiring Explanation:

Relay to ESP32:

IN pin (Relay) → GPIO (e.g., D23) on ESP32

This is the control signal pin from ESP32 to the relay.

VCC (Relay) → 5V on ESP32

Powers the relay module.

GND (Relay) → GND on ESP32

Common ground reference.

Lamp to Relay:

One terminal of the AC line connects to the Common (COM) of the relay.

The Normally Open (NO) terminal connects to one side of the lamp.

The other side of the lamp goes to AC Neutral

Note: This part must be handled with caution. Always use proper insulation and follow electrical safety protocols.

How It Works:

1. Google Assistant hears your voice command (e.g., "Turn on the lamp").

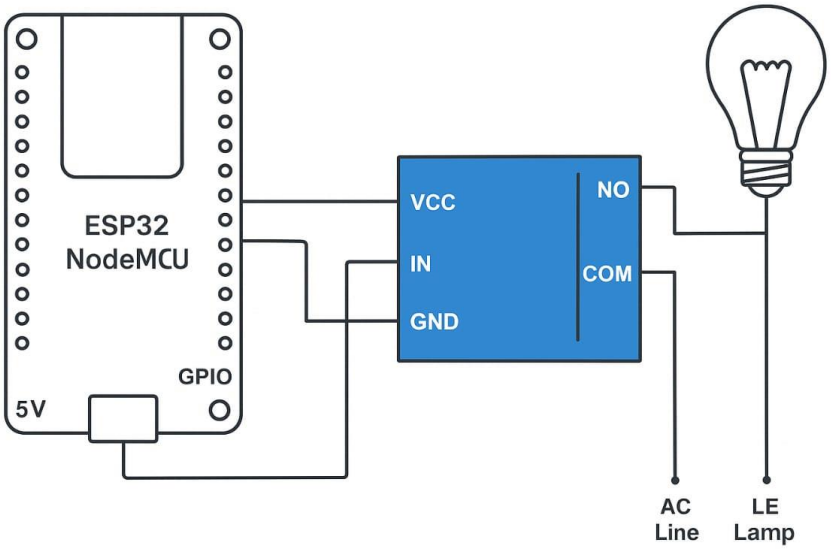
2. Sinric Pro forwards that command to the cloud and then to

your ESP32 via Wi-Fi.

3. ESP32 receives the command and sets the GPIO pin HIGH or LOW.

4. This toggles the relay, switching the lamp ON or OFF.

3.1 ESP32/NODEMCU CIRCUIT



1. ESP32 NodeMCU

The main microcontroller with built-in Wi-Fi, programmed to control the relay based on commands (e.g., from Google Assistant via Sinric Pro).

2. Relay Module (5V)

Electrically isolated switch that can turn high-voltage AC devices ON or OFF.

3. LED Lamp (AC-powered)

Connected to the relay, this is the final load to be controlled.

Connections:

ESP32 to Relay Module:

5V → VCC (Powers the relay)

GND → GND (Common ground)

GPIO (e.g., D23) → IN (Control signal)

Relay to AC LED Lamp:

COM (Common) connects to AC live wire

NO (Normally Open) connects to one terminal of the LED lamp

The other terminal of the LED lamp connects to the AC neutral line

How It Works:

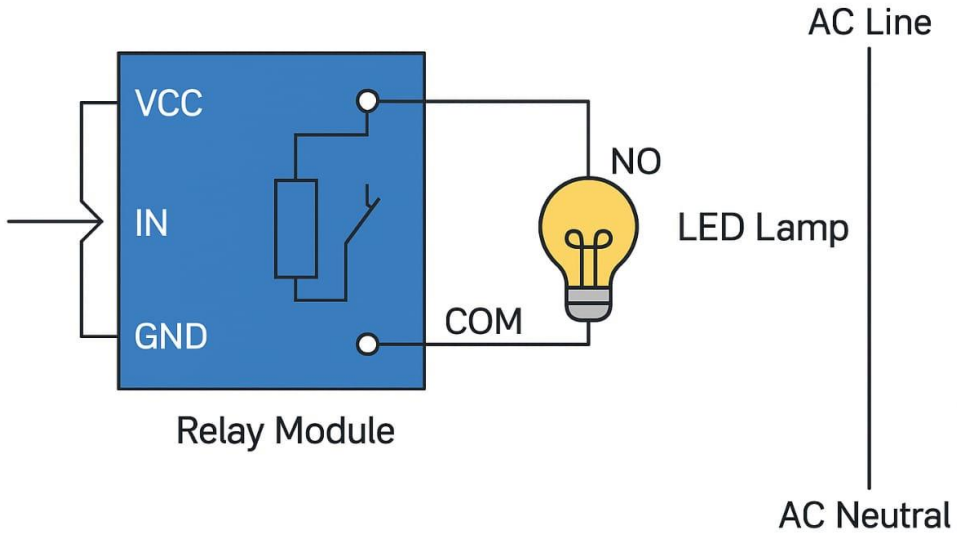
1. When the ESP32 sends a HIGH signal to the IN pin of the relay, it closes the contact between COM and NO.

2. This completes the AC circuit and turns ON the LED lamp.

3. When the signal is LOW, the relay opens, cutting power to the lamp and turning it OFF.

3.2 RELAY AND LED INTERFACE

Here's how the relay module interfaces with an LED lamp for control via an ESP32:



Interface Overview:

1. Relay Module Pins:

VCC – Power supply (typically 5V from ESP32)

GND – Ground

IN – Control signal from ESP32 GPIO

COM (Common) – Common terminal of the relay switch

NO (Normally Open) – Connects to lamp (circuit closes when relay is activated)

NC (Normally Closed) – (Unused in this case)

Circuit Flow:

A. Control Side (Low Voltage):

ESP32 sends a signal from a GPIO pin to the IN pin of the relay.

If the pin is set LOW or HIGH (depending on module logic), the relay activates (electromagnetic coil pulls switch).

B. Load Side (High Voltage):

AC Line → COM

NO → Lamp → AC Neutral

When the relay is activated:

COM and NO are connected, completing the AC circuit.

The LED lamp turns ON.

When deactivated:

COM and NO are disconnected.

The lamp turns OFF.

Interface Logic:

Relay acts as a physical switch.

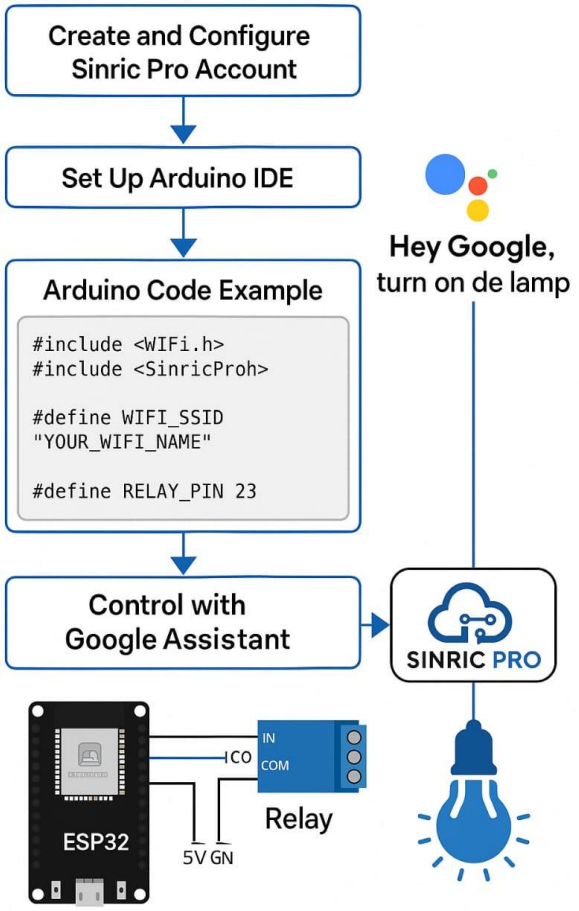
Isolated from ESP32 via optocoupler (in most relay modules) for safety.

Only ESP32 needs to switch the IN pin logic; the relay does the actual high-voltage switching.

CHAPTER 4

SOFTWARE IMPLEMENTATION

Here is a step-by-step software implementation guide to control an LED lamp using Sinric Pro, ESP32, and Google Assistant:



Step 1: Create and Configure Sinric Pro Account

1. Go to https://portal.sinric.pro

2. Create an account and log in.

3. Click “+ Add Device”, choose Switch (for controlling an LED lamp).

4. Note the following:

Device ID

App Key

App Secret

Step 2: Set Up Arduino IDE

1. Install Arduino IDE from https://www.arduino.cc

2. Install ESP32 Board Support from Board Manager

3. Install the following libraries via Library Manager:

SinricPro

WiFi

Step 3: Arduino Code Example

```
#include <WiFi.h>

#include <SinricPro.h>

#include <SinricProSwitch.h>
```

```
#define WIFI_SSID         "YOUR_WIFI_NAME"
#define WIFI_PASSWORD     "YOUR_WIFI_PASSWORD"
#define APP_KEY           "YOUR_SINRICPRO_APP_KEY"
#define APP_SECRET        "YOUR_SINRICPRO_APP_SECRET"
#define SWITCH_ID         "YOUR_DEVICE_ID"   // From Sinric Pro
#define RELAY_PIN         23                 // GPIO connected to relay module

void setupWiFi() {
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi connected");
}

bool onPowerState(const String &deviceId, bool &state) {
  digitalWrite(RELAY_PIN, state ? HIGH : LOW);
```

```
Serial.printf("Device %s turned %s\r\n", deviceId.c_str(), state ?
"on" : "off");
  return true;
}
void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);
  setupWiFi();
  SinricProSwitch &mySwitch = SinricPro[SWITCH_ID];
  mySwitch.onPowerState(onPowerState);
  SinricPro.begin(APP_KEY, APP_SECRET);
  SinricPro.restoreDeviceStates(true);
}
void loop() {
  SinricPro.handle();
}
```

Step 4: Control with Google Assistant

Option A: Native Google Smart Home Integration (Premium on Sinric Pro)

Go to Sinric Pro → Smart Home → Google

Follow instructions to link Sinric Pro to Google Home

After linking, the device (e.g., "Lamp") will appear in your Google Home app

Use voice: “Hey Google, turn on the lamp”

Option B: Use IFTTT (Free)

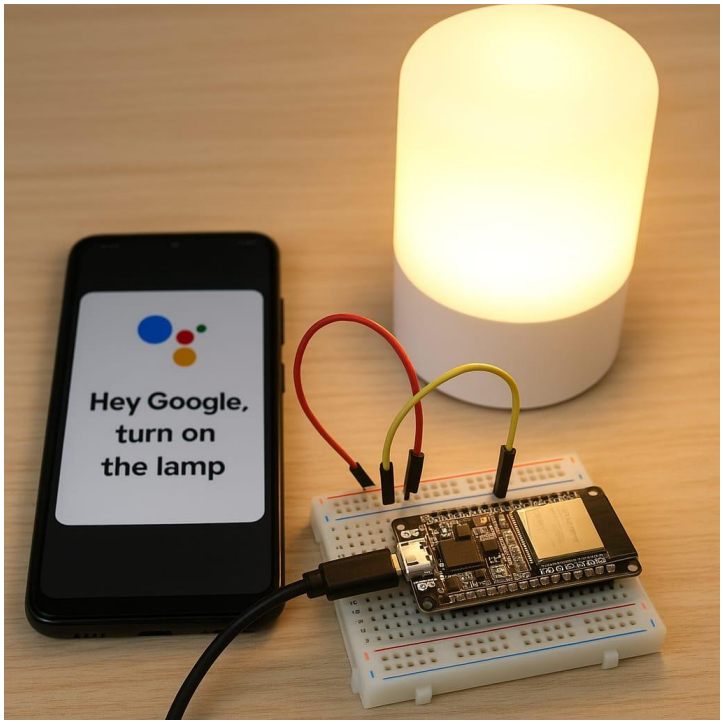
1. Create an IFTTT account

2. Connect Google Assistant and Webhooks

3. Trigger IFTTT applet to call Sinric Pro’s REST API to toggle the device

4.1 SINRIC PRO SETUP

To set up SinricPro for controlling devices like an LED lamp with an ESP32, follow these detailed steps:



Step-by-Step SinricPro Setup:

1. Create a SinricPro Account:

Visit the SinricPro website and sign up for a new account.

After creating an account, log in to the SinricPro Dashboard.

2. Create a New Device:

Once logged in, click "Add Device" in the SinricPro Dashboard.

Select the Smart Switch (or another device type if you're using different hardware, e.g., Smart Light).

Name your device (e.g., "LED Lamp")

Click Create to generate the device.

3. Obtain the API Key and Device ID:

After creating the device, you'll be directed to the device details page.

Copy the API Key and Device ID from this page. You'll need them for the ESP32 Arduino code.

4. Set Up Google Assistant:

In the SinricPro Dashboard, click on the "Google Assistant" tab.Follow the instructions to link your SinricPro account with Google Assistant. You may need to sign into your Google account.

Once linked, you can add your SinricPro device (LED Lamp) to Google Assistant by saying, "Hey Google, sync my devices".

Your device will show up in the Google Home app.

5. Install Libraries in Arduino IDE:

SinricPro Library: Open Arduino IDE, go to Sketch > Include Library > Manage Libraries, search for SinricPro, and install it.

WiFi Library: Ensure that the WiFi library for ESP32 is installed. (This should already be part of the ESP32 core libraries.)

6. Write Code for the ESP32:

Use the example code provided below to control your LED lamp.

```
#include <WiFi.h>

#include <SinricPro.h>

#include <SinricProSwitch.h>

// Replace with your network credentials

const char* ssid = "YOUR_SSID";

const char* password = "YOUR_PASSWORD";

// Replace with your SinricPro API details

const char* apiKey = "YOUR_API_KEY";

const char* deviceId = "YOUR_DEVICE_ID";

#define LED_PIN 5  // Pin where the LED is connected

// Create SinricProSwitch object for controlling the device
```

```
SinricProSwitch mySwitch(deviceId);
void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
// Connect to Wi-Fi
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");
  // Connect to SinricPro
  SinricPro.begin(apiKey);
  // Set callback for power state change (turn on/off)
  mySwitch.onPowerState(onPowerState);
}
```

```
void loop() {
  SinricPro.handle(); // Handle SinricPro communication
}
// Function to handle the power state of the device (on/off)
bool onPowerState(const String &deviceId, bool state) {
if (state) {
    digitalWrite(LED_PIN, HIGH);  // Turn on LED
  } else {
    digitalWrite(LED_PIN, LOW);   // Turn off LED
  }
  return true;
}
```

7. Upload the Code to ESP32:

Connect your ESP32 to your computer.

Select ESP32 as the board in Tools > Board in Arduino IDE.

Select the correct COM port in Tools > Port.

Upload the code to the ESP32.

8. Test the Device:

Open Google Home and say, "Hey Google, sync my devices."

After syncing, you should be able to control your LED lamp using Google Assistant with commands like:

"Hey Google, turn on the LED."

"Hey Google, turn off the LED.”

This completes the SinricPro setup for controlling an LED lamp (or other devices) using the ESP32. You can now control your devices via Google Assistant!

4.2 GOOGLE ASSISTANT INTEGRATION

Google Assistant integration allows users to control devices using voice commands. In IoT projects, platforms like SinricPro simplify this by acting as a bridge between Google Assistant and hardware like the ESP32. By creating a SinricPro account, adding a device (e.g., switch for LED), uploading ESP32 code with Wi-Fi and API credentials, and linking SinricPro to Google Home, users can control their devices using voice commands such as "Hey Google, turn on the lamp." This setup enables smart automation using minimal effort



SinricPro: Connects ESP32 to Google Assistant easily. ESP32: Ideal for smart projects; has Wi-Fi built-in. Google Home App: Needed to link SinricPro and control devices.Device ID & API Key: Must be copied from SinricPro dashboard. Voice Control: Use simple commands like “Turn on light.”

CHAPTER 5

CODE FLOW AND LOGIC

Code Flow & Logic

1. Setup Section:

Initialize Serial communication for debugging.

Set LED pin mode (e.g., pinMode(LED_PIN, OUTPUT)).

Connect to Wi-Fi using WiFi.begin().

Wait until connected (WiFi.status() loop).

2. SinricPro Setup:

Initialize SinricPro with SinricPro.begin(API_KEY).

Create a Switch object using your Device ID.

Attach a callback function using onPowerState() to handle on/off commands.

3. Loop Section:

Continuously call SinricPro.handle() to maintain WebSocket communication with SinricPro server.4. Callback Function Logic:

onPowerState() receives a boolean state (true for ON, false for OFF).

Use digitalWrite(LED_PIN, state ? HIGH : LOW) to control the LED. Return true to acknowledge the action.

Simplified Logic Flow:

1. Google Assistant voice command
2. SinricPro server receives it
3. SinricPro sends ON/OFF signal to ESP32
4. ESP32 handles it via onPowerState()
5. LED turns ON/OFF accordingly.

CHAPTER 6

PROGRAMMING AND IMPLEMENTATION

```
#include <WiFi.h>

#include <ArduinoJson.h>

#include "SinricPro.h"

#include "SinricProSwitch.h"

// WiFi Credentials

#define WIFI_SSID     "XXXXX"

#define WIFI_PASS     ",,,,,,,,"

// SinricPro Credentials

#define APP_KEY       "37e5df0e-cc51-41fe-83ca-3bb6f91f9940"

#define APP_SECRET    "60c05df5-f083-43f1-813f-
9e049fc7079a-08865ebf-52bb-43f4-b8f5-d31282e3e68e"

#define DEVICE_ID_1   "68179c12dc4a25d5c3c44459"

#define DEVICE_ID_2   "68179c4f8ed485694c2ce6e9"

#define DEVICE_ID_3   "68179c6bdc4a25d5c3c444fe"

#define DEVICE_ID_4   "DUMMY"
```

```
// Relay Pins
#define RELAY_1  5
#define RELAY_2  18
#define RELAY_3  19
#define RELAY_4  21
bool onPowerState(const String &deviceId, bool &state) {
  Serial.printf("Device %s turned %s\r\n", deviceId.c_str(), state ? "ON" : "OFF");
  if (deviceId == DEVICE_ID_1) digitalWrite(RELAY_1, !state);
  else if (deviceId == DEVICE_ID_2) digitalWrite(RELAY_2, !state);
  else if (deviceId == DEVICE_ID_3) digitalWrite(RELAY_3, !state);
  else if (deviceId == DEVICE_ID_4) digitalWrite(RELAY_4, !state);
  return true;
}
void setupWiFi() {
```

```
Serial.print("Connecting to WiFi...");
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Connected!");
}
void setupSinricPro() {
  SinricProSwitch &sw1 = SinricPro[DEVICE_ID_1];
  SinricProSwitch &sw2 = SinricPro[DEVICE_ID_2];
  SinricProSwitch &sw3 = SinricPro[DEVICE_ID_3];
SinricProSwitch &sw4 = SinricPro[DEVICE_ID_4];
  sw1.onPowerState(onPowerState);
  sw2.onPowerState(onPowerState);
  sw3.onPowerState(onPowerState);
  sw4.onPowerState(onPowerState);
  SinricPro.begin(APP_KEY, APP_SECRET);
```

```
SinricPro.restoreDeviceStates(true);
}
void setup() {
  Serial.begin(9600);
  pinMode(RELAY_1, OUTPUT); digitalWrite(RELAY_1, HIGH);
  pinMode(RELAY_2, OUTPUT); digitalWrite(RELAY_2, HIGH);
  pinMode(RELAY_3, OUTPUT); digitalWrite(RELAY_3, HIGH);
  pinMode(RELAY_4, OUTPUT); digitalWrite(RELAY_4, HIGH);
  setupWiFi();
  setupSinricPro();
}
void loop() {
  SinricPro.handle();
}
```

CHAPTER 7

RESULT AND DISCUSSION

Result:

The integration between Google Assistant, SinricPro, and ESP32 was successfully implemented. Voice commands issued via Google Assistant were able to turn the LED lamp ON and OFF in real-time. The ESP32 responded to cloud-based commands reliably over Wi-Fi, confirming that the SinricPro platform effectively bridged the communication.

Discussion:

This project demonstrated how cloud services like SinricPro can simplify the process of connecting microcontrollers to smart home assistants. The use of voice commands enhanced user convenience and showed potential for scaling to other appliances (e.g., fans, lights, or smart sockets). Real-time

responses and seamless Google Home integration were key successes.

However, a reliable internet connection is essential, and slight delays may occur due to cloud latency. Overall, this setup is a practical solution for low-cost home automation using widely available components.

CHAPTER 8

CONCLUSION

The project successfully demonstrated how to control an LED lamp using Google Assistant, ESP32, and SinricPro. By integrating cloud-based voice control with IoT hardware, a simple yet effective smart automation system was built. This approach highlights the potential for expanding to larger home automation setups using affordable components and minimal coding. The project also reinforces the importance of reliable internet connectivity and secure API usage in IoT solution.

CHAPTER 9

REFERENCES

1. SinricPro Official Website – https://sinric.pro

2. Google Assistant Developer Guide – https://developers.google.com/assistant

3. Arduino IDE – https://www.arduino.cc/en/software